SIMILARITY SEARCH

Full Detailed Professional Notes

Vectors, L2 Distance, Dot Product, Cosine Similarity, Metric Selection, Practical Considerations, Summary and Questions with Answers

Learning objectives
Explain similarity search, interpret vectors and norms, calculate L2 distance, dot product and cosine similarity, select an appropriate metric, and discuss production considerations for high-dimensional retrieval systems.

1. Similarity Search Overview

Similarity search finds the items in a dataset that most closely resemble a query item. Instead of requiring exact equality, it ranks candidates according to a mathematical distance or similarity score.

This capability is central to modern AI systems because text, images, videos, audio, products and users can be represented as numerical vectors. The same retrieval pattern can therefore support many data types.

Common applications

- Recommendation systems, such as finding movies or products related to a user preference
- Image and video retrieval based on visual content rather than filenames
- Natural language processing, including document, sentence and semantic search
- Biometric matching, including facial or feature-vector comparison
- Retrieval-augmented generation, where relevant text chunks are retrieved before an AI response is generated

Distance versus similarity

Measure	Interpretation
Distance	Lower values usually indicate that vectors are closer and therefore more similar.
Similarity	Higher values usually indicate that vectors are more alike.

> Core idea
> A distance function measures separation. A similarity function measures likeness. Many systems convert one form into the other, but the exact conversion depends on the selected metric and database API.

2. Mathematical Background

The following concepts explain why similarity metrics behave differently. In practice, vector libraries and databases perform these calculations automatically, but understanding the mathematics helps with metric selection and troubleshooting.

2.1 Cosine of an angle

For a right-angle triangle, the cosine of an angle is the ratio of the adjacent side to the hypotenuse.

```
cos(alpha) = adjacent / hypotenuse
```

If the adjacent side has length ||c|| and the hypotenuse has length ||a||, then:

```
cos(alpha) = ||c|| / ||a||
||c|| = ||a|| cos(alpha)
```

This projection relationship becomes important when deriving the geometric form of the dot product.

2.2 What is a vector?

A vector is a geometric object with both direction and magnitude. A vector with two components can be plotted on a two-dimensional Cartesian plane, while a three-component vector can be plotted in three dimensions.

```
a = [4, 8]
```

The value 4 is the x-component and 8 is the y-component. The vector itself is not a scalar length. Its length must be calculated separately.

2.3 Vector magnitude and the L2 norm

The magnitude of an n-dimensional vector is calculated using the Euclidean, or L2, norm:

```
||a|| = sqrt(sum from k=1 to n of a_k^2)
```

For a = [4, 8]:

```
||a|| = sqrt(4^2 + 8^2) = sqrt(80) approximately 8.94
```

For embeddings, vector direction often captures semantic orientation or topic. Magnitude may sometimes represent strength, confidence, popularity or another learned signal. This interpretation is model-dependent and should be verified for the embedding model in use.

2.4 Multiple vectors

Consider a = [4, 8] and b = [11.5, 5]. A metric provides a score describing how close or aligned these vectors are. Different metrics emphasize different properties and may therefore rank the same candidates differently.

3. L2 Distance, also called Euclidean Distance

L2 distance measures the straight-line distance between two vector endpoints in Euclidean space.

```
L2(a,b) = sqrt(sum from i=1 to n of (a_i - b_i)^2)
```

Example 1: two-dimensional vectors

For a = [4, 8] and b = [11.5, 5]:

```
L2(a,b) = sqrt((4 - 11.5)^2 + (8 - 5)^2)
= sqrt(56.25 + 9)
= sqrt(65.25)
approximately 8.08
```

Example 2: three-dimensional vectors

For q = [1, 2, 3] and r = [4, 5, 6]:

```
L2(q,r) = sqrt((1 - 4)^2 + (2 - 5)^2 + (3 - 6)^2)
= sqrt(9 + 9 + 9)
= sqrt(27)
approximately 5.20
```

Properties

- Represents geometric straight-line separation.
- Sensitive to changes in both vector magnitude and direction.
- Works naturally when absolute position and scale are meaningful.
- Can be affected by feature ranges, so preprocessing or feature scaling may be important.
- In very high-dimensional spaces, distance discrimination may weaken because many candidates can appear similarly distant.

Typical use cases

- Spatial or geometric data

- Computer-vision feature matching where Euclidean geometry is meaningful
- Clustering and nearest-centroid methods
- Low- or moderate-dimensional continuous feature spaces

> Selection guidance
> Use L2 distance when the absolute placement and scale of vectors carry information. Do not select it only because it is intuitive; confirm that the embedding model was trained or documented for Euclidean comparison.

4. Dot Product, also called Inner Product

The dot product combines corresponding components and sums the results. It measures alignment while retaining the effect of vector magnitude.

```
a dot b = sum from i=1 to n of a_i b_i
```

Example 1

For a = [4, 8] and b = [11.5, 5]:

```
a dot b = (4 x 11.5) + (8 x 5)
= 46 + 40
= 86
```

Example 2

For q = [1, 2, 3] and r = [4, 5, 6]:

```
q dot r = (1 x 4) + (2 x 5) + (3 x 6)
= 4 + 10 + 18
= 32
```

Geometric interpretation

The dot product can also be written using vector magnitudes and the angle alpha between them:

```
a dot b = ||a|| ||b|| cos(alpha)
```

This formula shows that the score increases when vectors are well aligned and when their lengths are large. The projection of a onto b has length ||a|| cos(alpha); multiplying that projection length by ||b|| gives the dot product.

Alternative calculation for the example

For a = [4, 8], b = [11.5, 5] and alpha approximately 39.94 degrees:

```
||a|| approximately 8.94
||b|| = sqrt(11.5^2 + 5^2) approximately 12.54
cos(39.94 degrees) approximately 0.767
a dot b approximately 8.94 x 12.54 x 0.767 approximately 85.99
```

The small difference from 86 is caused by rounding.

Interpretation and properties

- A positive value indicates an angle below 90 degrees.
- A zero value indicates orthogonality.
- A negative value indicates an angle above 90 degrees.

- Larger values can result from stronger alignment, larger magnitudes, or both.
- The calculation maps efficiently to matrix operations and hardware acceleration.

Typical use cases

- Recommendation systems when magnitude contains a useful signal such as learned preference strength or item prominence
- Neural-network operations and matrix factorization
- Ranking systems whose model was trained with inner-product scoring

> Important caution
> A larger dot product is not automatically better in every model. If unbounded magnitude is accidental rather than meaningful, high-norm vectors can dominate the ranking. Follow the embedding model specification and consider normalization when appropriate.

5. Cosine Similarity and Cosine Distance

Cosine similarity measures the cosine of the angle between two vectors. It focuses on direction and removes the effect of vector length.

```
cosine_similarity(a,b) = (a dot b) / (||a|| ||b||)
```

For non-zero vectors, cosine similarity normally ranges from -1 to 1: 1 means the same direction, 0 means perpendicular directions, and -1 means opposite directions.

Worked example

Using a dot b = 86, ||a|| approximately 8.94 and ||b|| approximately 12.54:

```
cosine_similarity(a,b) = 86 / (8.94 x 12.54)
approximately 0.767
```

Cosine distance

A frequently used conversion is:

```
cosine_distance(a,b) = 1 - cosine_similarity(a,b)
For the example: 1 - 0.767 = 0.233
```

Vector normalization

A vector is normalized by dividing each component by its L2 norm:

```
norm(a) = a / ||a||
```

For a = [4, 8] and ||a|| approximately 8.94:

```
norm(a) = [4/8.94, 8/8.94]
approximately [0.448, 0.895]
```

A normalized vector has unit length. The sum of its squared components is approximately 1. For unit vectors, cosine similarity equals the dot product:

```
cosine_similarity(a,b) = norm(a) dot norm(b)
```

Properties

- Focuses on orientation rather than magnitude.
- Invariant to positive rescaling of a vector.

- Common for high-dimensional text and semantic embeddings.
- Can be computed efficiently as a dot product when vectors are normalized.
- Is undefined for a zero vector because its norm is zero.

Typical use cases

- Semantic document and sentence search
- Natural language processing and text embeddings
- RAG retrieval when the embedding model recommends cosine comparison
- Content matching when vector length should not influence the ranking

> Operational guidance
> Normalize consistently. Comparing normalized query vectors with unnormalized stored vectors, or mixing model versions, can produce misleading results.

6. Metric Comparison and Selection

Metric	Magnitude sensitive	Score preference	Best fit
L2 distance	Yes	Lower is closer	Spatial, geometric and scale-aware features
Dot product	Yes	Higher is more similar	Recommendations or ranking when magnitude matters
Cosine similarity	No	Higher is more similar	Semantic text and directional embeddings
Cosine distance	No	Lower is closer	Distance-style APIs using angular similarity

Decision framework

- Start with the embedding model documentation. The metric used during training or evaluation is normally the safest default.
- Use cosine similarity when direction represents meaning and magnitude should be ignored.
- Use dot product when magnitude is intentionally learned and contributes to relevance.
- Use L2 when absolute coordinate separation and feature scale are meaningful.
- Validate the choice with representative queries and relevance judgments instead of relying only on theoretical expectations.

Golden rule

> Quick selection
> Meaning and direction dominate -> cosine similarity. Magnitude and direction both matter -> dot product. Absolute geometric distance matters -> L2 distance.

Equivalent rankings under normalization

For unit-normalized vectors, dot-product similarity and cosine similarity are equal. L2 distance is also monotonically related to cosine similarity for unit vectors: minimizing squared L2 distance produces the same ranking as maximizing cosine similarity. Database score values will differ, but the neighbor order can match.

7. Practical and Production Considerations

7.1 Normalization

- Normalize vectors when cosine-based comparison is intended and the model does not already produce unit vectors.
- Apply the same preprocessing to stored vectors and query vectors.
- Avoid assuming normalization. Verify it by calculating sample norms or checking model documentation.

7.2 High-dimensional data

- As dimensionality grows, distance distributions can become less discriminative.
- Cosine comparison is common for high-dimensional text embeddings, but it is not universally superior.
- Dimensionality reduction may reduce storage and search cost, but it can also discard information and should be evaluated.

7.3 Exact versus approximate search

Exact nearest-neighbor search compares the query with all eligible vectors and returns mathematically exact results. Approximate nearest-neighbor methods trade a small amount of recall for much lower latency and better scalability. Common vector indexes use graph-, partition- or quantization-based strategies.

7.4 Index and metric compatibility

The selected index must support the intended metric. Some platforms implement cosine search by normalizing vectors and using an inner-product index. Changing metrics may require rebuilding the index.

7.5 Metadata filtering

Production retrieval commonly combines vector similarity with structured conditions, such as tenant, language, document category, security label, date, product availability or geographic boundary. Filtering strategy can materially affect performance and relevance.

7.6 Thresholds and top-k

- Top-k retrieval returns a fixed number of candidates even when some are weak matches.
- A similarity or distance threshold rejects candidates below an acceptable relevance level.
- Thresholds are metric- and model-specific; they should be calibrated using representative data.

7.7 Model and vector consistency

- Do not compare vectors produced by unrelated embedding models.
- Track model name, version, dimensionality and preprocessing configuration.
- A model upgrade typically requires controlled re-embedding, validation and re-indexing.

7.8 Security and governance

- Apply authorization before returning source content.
- Preserve tenant and document-level access boundaries.
- Avoid exposing sensitive metadata through nearest-neighbor results.
- Log model, index and retrieval configuration for reproducibility and incident analysis.

7.9 Monitoring

- Measure latency, throughput, index freshness and failure rates.
- Evaluate relevance using labeled queries, recall-at-k, precision-at-k or task-specific success measures.
- Monitor score distributions and retrieval drift after model or data changes.

> Production principle
> Fast retrieval is not enough. A reliable system must also provide relevant, authorized, observable and reproducible results.

8. Enterprise Examples

Semantic knowledge search

Documents are divided into chunks, converted into embeddings and stored with metadata. A user query is embedded with the same model. Similarity search retrieves candidates, after which security filters and reranking can be applied.

Recommendation service

User and item representations are compared to retrieve candidate products, movies or services. Dot product can be appropriate when the model intentionally encodes both affinity and magnitude. Business rules then remove unavailable or ineligible items.

Image matching

A vision model converts each image into a feature vector. L2 or cosine comparison retrieves visually similar images, depending on the model design. Metadata filters can limit results to an authorized collection.

Biometric matching

A biometric template is compared with enrolled templates. This is a high-risk domain requiring validated thresholds, anti-spoofing controls, privacy safeguards and human review where required.

RAG retrieval

A question embedding is compared with document-chunk embeddings. The chosen metric retrieves relevant context, while metadata filters, reranking and source attribution improve grounding quality.

> Architecture flow
> Source data -> preprocessing -> embedding model -> vector and metadata storage -> query embedding -> filtering -> nearest-neighbor search -> reranking -> authorized result or downstream AI application

9. Final Summary

Similarity search retrieves the nearest or most related items by comparing vector representations. Its effectiveness depends on the interaction between the embedding model, distance metric, vector preprocessing, index type, metadata filtering and evaluation method.

- L2 distance measures absolute Euclidean separation and is sensitive to magnitude and direction.
- Dot product rewards alignment and magnitude, making it useful where vector length carries a deliberate signal.
- Cosine similarity measures direction while ignoring magnitude and is common for semantic and text embeddings.
- Normalization converts vectors to unit length and makes cosine similarity equal to the dot product.
- For unit vectors, cosine similarity, dot product and L2 can yield equivalent rankings even though their scores differ.
- Production systems should validate relevance, enforce authorization, monitor retrieval quality and manage model or index versions carefully.

> Main takeaway
> Choose the metric that matches how the embedding model represents similarity. Then verify the choice empirically with real enterprise queries and controlled relevance evaluation.

10. Questions with Answers

1. What is similarity search?

Answer: It is the retrieval of items that are closest or most alike to a query according to a distance or similarity metric.

2. Why do vector databases use similarity search?

Answer: Embeddings encode data as vectors, so mathematical comparison can retrieve related items even when exact values or keywords differ.

3. What is a vector?

Answer: A vector is an ordered set of numerical components that can be understood geometrically as having direction and magnitude.

4. What is vector magnitude?

Answer: It is the vector length. The L2 norm calculates it as the square root of the sum of squared components.

5. What is L2 distance?

Answer: It is the straight-line Euclidean distance between two vector endpoints. Lower values indicate closer vectors.

6. When is L2 distance appropriate?

Answer: When absolute geometry, position and scale are meaningful and the embedding or feature model supports Euclidean comparison.

7. What is the dot product?

Answer: It is the sum of the products of corresponding vector components. It reflects both directional alignment and magnitude.

8. Why can dot product suit recommendation systems?

Answer: If the learned vector magnitudes intentionally represent preference strength, popularity or another useful signal, dot product retains that information.

9. What is cosine similarity?

Answer: It is the dot product divided by the product of vector magnitudes, measuring the cosine of the angle between vectors.

10. What does cosine similarity ignore?

Answer: It removes the effect of vector length and focuses on direction, provided the vectors are non-zero.

11. What is cosine distance?

Answer: A common definition is one minus cosine similarity. Lower values indicate greater similarity.

12. What is vector normalization?

Answer: It is scaling a vector by its L2 norm so that the resulting vector has unit length.

13. Why normalize embeddings?

Answer: Normalization removes magnitude effects and allows cosine similarity to be computed as a dot product.

14. Are cosine similarity and dot product always equal?

Answer: No. They are equal when both vectors are unit-normalized.

15. Can L2 and cosine return the same ranking?

Answer: Yes. For unit-normalized vectors, minimizing L2 distance is equivalent to maximizing cosine similarity, so rankings can match.

16. What is the curse of dimensionality?

Answer: It describes problems that emerge in high-dimensional spaces, including reduced contrast between distances and increased search complexity.

17. Which metric is best for RAG?

Answer: There is no universal metric. Use the metric recommended for the selected embedding model, then validate it with representative retrieval tests. Cosine is common for text embeddings.

18. Which metric is computationally efficient?

Answer: Dot product maps efficiently to matrix operations. For normalized vectors it also computes cosine similarity.

19. What is top-k retrieval?

Answer: It returns the k highest-similarity or lowest-distance candidates according to the metric.

20. Why use a score threshold?

Answer: A threshold prevents weak matches from being returned only because the system must fill the requested top-k count.

21. What is approximate nearest-neighbor search?

Answer: It uses an index to retrieve highly likely neighbors faster than exhaustive comparison, usually with a trade-off between latency, memory and recall.

22. Why are metadata filters important?

Answer: They constrain retrieval by business, security or content conditions such as tenant, language, access level or item availability.

23. Can vectors from different models be compared?

Answer: Not reliably. Different models can produce incompatible vector spaces, dimensions and similarity behavior.

24. What must be managed during a model upgrade?

Answer: Model version, dimensionality, normalization, re-embedding, index rebuild, relevance validation and rollout or rollback controls.

25. What should a production similarity service monitor?

Answer: Latency, throughput, index freshness, failures, score distributions, relevance metrics, authorization correctness and drift after changes.

11. Review Questions for Further Study

Beginner

- Calculate the L2 norm of [3, 4].
- Explain the difference between a similarity score and a distance score.
- Describe what cosine similarity measures.

Intermediate

- Explain why normalization makes cosine similarity equal to dot product.
- Describe a case where dot product and cosine similarity rank candidates differently.
- Explain why a top-k result may still contain irrelevant items.

Advanced

- Design a metric-evaluation plan for an enterprise RAG system.
- Explain the latency, recall and memory trade-offs of approximate nearest-neighbor indexes.
- Describe a zero-downtime re-embedding and index migration strategy.
- Explain how metadata filtering and authorization should interact with vector retrieval.

Study use
This document is structured for technical learning, certification review, AI architecture discussions, RAG design and interview preparation.